API Requests & The V8 Engine

The Beginner's Guide to Network Requests and Browser Mechanics

1. What exactly is an API?

Imagine you are sitting at a restaurant table with a menu. You want to order a burger. You cannot walk directly into the kitchen (the server/database) and cook it yourself. You need a messenger.

The **Waiter** takes your order, brings it to the kitchen, and then carries your food back to your table. In web development, the **API (Application Programming Interface)** is that waiter. It is the messenger that takes your JavaScript request, carries it over the internet to a server (like GitHub or Instagram), and brings the data back to your screen.

2. The V8 Engine: The Translator

Computers do not understand JavaScript. They only understand electricity—specifically, binary machine code (0s and 1s). So, how does your browser run your JS code? It uses an **Engine**. The most famous one is Google's **V8 Engine**.



⚠️ Mind-Blowing Fact: console.log is NOT JavaScript!

Here is a secret that surprises many beginners: `console.log()`, `document.getElementById()`, and `setTimeout()` are **not** part of the JavaScript language. If you read the official JS manual, they don't exist!

They are "Web APIs." The browser (using C++ code) injects these super-powers into your JavaScript environment. V8 acts as the bridge connecting your JS code to the browser's internal features.

3. How Data Travels: Introducing JSON

When our "API Waiter" brings data back from the server, it cannot bring a native JavaScript Object. Why? Because the internet only knows how to transmit pure text.

To solve this, we use **JSON (JavaScript Object Notation)**. JSON is simply a way of taking a complex object and turning it into a giant, flat text string so it can safely travel through internet cables.

```
      // 1. This is a real JS Object (Only works inside your code) const user = { name: "Hitesh", followers: 500 }; // 2. This is JSON (It is just a giant string of text) // '{"name": "Hitesh", "followers": 500}'
```

4. The Old-School Way: XMLHttpRequest (XHR)

Today, web developers mostly use the modern `fetch()` command to talk to APIs. However, to truly master JavaScript, you must learn what `fetch` is hiding from you.

Before `fetch`, developers used an object called `XMLHttpRequest` (often abbreviated as XHR or AJAX). It is the raw, mechanical engine of internet requests.

> 💡 **Why study the old XHR method?**
>
> Imagine learning to drive. `fetch()` is an automatic transmission car—it shifts gears for you, making it easy. `XMLHttpRequest` is a manual transmission car. It forces you to understand the clutch, the gears, and the engine. Learning XHR makes you a much stronger debugger because you understand the exact microscopic steps a network request takes.

5. The 5 Stages of an API Request (Ready States)

When you send an XHR request, it doesn't happen instantly. It travels through 5 distinct phases, numbered 0 to 4. We call these the **Ready States**.

State	Name	The Mailbox Analogy (What it means)
0	`UNSENT`	You wrote the letter, but it is still sitting on your desk. (You created the request, but haven't configured it).
1	`OPENED`	You put the stamp on and addressed it. (You used `.open()` to tell JS where to send it).
2	`HEADERS_RECEIVED`	You dropped it in the mailbox, and the post office scanned it. (The server said "I got it!").
3	`LOADING`	The mailman is currently handing you the return package, but it's only halfway out of his bag. (Data is downloading).
4	`DONE`	The package is fully in your hands and you have opened it. (The request is complete and data is ready to use).

> ⚠️ **The Golden Rule of API Data**
> You should **never** try to use the data during states 1, 2, or 3. If you try to read the data while it is still `LOADING`, your code will crash or return half-finished gibberish. Always write an `if` statement to check if the state is exactly `4` before doing anything.

6. Listening to State Changes

To watch the request move through these 5 stages, JavaScript provides a special event listener called `onreadystatechange`. It fires automatically every time the state number changes.

```
      const xhr = new XMLHttpRequest(); // This function runs every time the state changes (0 -> 1 -> 2 -> 3 -> 4) xhr.onreadystatechange = function() {
console.log("Current State is: ", xhr.readyState); }
```

7. The Complete API Request Code

Here is how we put all the pieces together to fetch a user's GitHub profile data using the old-school XHR method.

```
      const requestUrl = 'https://api.github.com/users/hiteshchoudhary'; // 1. Create the engine object const xhr = new XMLHttpRequest(); // 2. Open the connection (Method, URL) -> Moves to State 1 xhr.open('GET', requestUrl); // 3. Listen for changes xhr.onreadystatechange = function() { // 4. ONLY do work if the state is 4 (DONE) if (xhr.readyState === 4) { // 5. The data arrives as a String. We MUST convert it back to an Object! const data = JSON.parse(this.responseText); console.log("Followers count: ", data.followers); } } // 6. Finally, push the send button! xhr.send();
```

8. The String Trap: `JSON.parse()`

Look closely at step 5 in the code above. The data we get back is stored in `this.responseText`. Remember how the internet only sends text (JSON)?

If we try to ask a string of text for its `followers`, JavaScript will say `undefined`. We must use `JSON.parse()` to transform that text back into a real JavaScript Object.

```
      // WRONG WAY: console.log(this.responseText.followers); // Returns undefined // RIGHT WAY: const realObject = JSON.parse(this.responseText); console.log(realObject.followers); // Returns the actual number
```

💡 Why use `this` and not Arrow Functions?

In the code above, we use a normal `function() {}` instead of a modern arrow function `() => {}`. Why? Because in XHR, the keyword `this` points to the specific XHR object running the code. Arrow functions **do not have their own `this`**. If you use an arrow function here, `this.responseText` will fail to find your data.